

TP 1 – Python : quelle est la nature de ce quadrilatère ?

Seconde – Géométrie repérée

Ce document reproduit le fichier `tp1-quadrilateres-seconde.py`, à ouvrir avec Spyder.

```

1 # =====
2 #           TP 1 - PYTHON : QUELLE EST LA NATURE DE CE QUADRILATÈRE ?
3 #           Seconde - Géométrie repérée
4 # =====
5 #
6 # OBJECTIF
7 #   Écrire, petit à petit, des fonctions qui reçoivent les coordonnées des
8 #   sommets d'un quadrilatère ABCD et qui permettent de dire s'il s'agit
9 #   d'un carré, d'un rectangle, d'un losange, d'un parallélogramme, ou d'un
10 #   quadrilatère quelconque.
11 #
12 # COMMENT EXÉCUTER UNE FONCTION AVEC SPYDER
13 #   Ce fichier est découpé en CELLULES : chaque cellule commence par une
14 #   ligne # %% et contient en général une fonction. On exécute une seule
15 #   cellule à la fois :
16 #
17 #   - Méthode 1 (conseillée) : cliquez n'importe où dans la cellule de la
18 #     fonction (elle est alors surlignée), puis appuyez sur Ctrl + Entrée.
19 #     Seule cette cellule est exécutée.
20 #   - Méthode 2 : sélectionnez à la souris toutes les lignes de la fonction
21 #     (de la ligne def jusqu'à sa dernière ligne), puis appuyez sur F9.
22 #
23 #   N'UTILISEZ PAS la touche F5 ni le triangle vert « Exécuter le fichier » :
24 #   ils exécutent TOUT le fichier, et la moindre erreur dans une fonction pas
25 #   encore terminée affiche un message d'erreur et arrête tout.
26 #
27 #   - Exécutez les cellules DANS L'ORDRE : une fonction n'existe pour Python
28 #     qu'après avoir exécuté sa cellule.
29 #   - Si vous modifiez une fonction, exécutez de nouveau sa cellule !
30 #
31 # COMMENT TESTER UNE FONCTION
32 #   Après avoir exécuté la cellule d'une fonction, tapez son nom avec des
33 #   valeurs dans la CONSOLE (en bas à droite), puis appuyez sur Entrée.
34 #   Exemple : tapez reste(7) puis Entrée.
35 #   Sous chaque fonction, vous trouverez des tests à taper, avec la réponse
36 #   que Python doit afficher. Si vous obtenez autre chose, corrigez !
37 #
38 # LÉGENDE
39 #   [EXEMPLE]      fonction déjà écrite : lisez-la, exécutez-la, testez-la.
40 #   [À COMPLÉTER]  remplacez les ... par ce qu'il faut.
41 #   [À ÉCRIRE]     écrivez la fonction entièrement, seul(e).
42 #   [QUESTION]     répondez dans le fichier, en commentaire (ligne avec #).
43 #
44 # Toutes les lignes qui commencent par # sont des COMMENTAIRES :
45 # Python les ignore, elles servent à expliquer le programme.
46 # =====
47
48
49 # %% [EXEMPLE 1] Le reste d'une division
50 #
51 # En Python, a % b donne le RESTE de la division de a par b.
52 # Par exemple 17 % 3 donne 2, car 17 = 3 × 5 + 2.
53
54 def reste(n):
55     r = n % 3
56     return(r)
57

```

```

58 # Tests à taper dans la console :
59 #     reste(17)                                réponse attendue : 2
60 #     reste(12)                                réponse attendue : 0
61 #
62 # [QUESTION 1] Que vaut le reste quand n est divisible par 3 ?
63 # Réponse :
64
65
66 # %% [EXEMPLE 2] Une fonction qui renvoie un message
67 #
68 # Rappel : if veut dire « si », else veut dire « sinon ».
69 # == veut dire « est égal à » (ne pas confondre avec = qui AFFECTE
70 # une valeur à une variable)
71 # Un texte s'écrit entre guillemets.
72
73 def divisible_par_3(n):
74     if n % 3 == 0:
75         return("divisible par 3")
76     else:
77         return("pas divisible par 3")
78
79 # Tests à taper dans la console :
80 #     divisible_par_3(12)                        réponse attendue : 'divisible par 3'
81 #     divisible_par_3(17)                        réponse attendue : 'pas divisible par 3'
82
83
84 # %% [EXEMPLE 3] Une fonction avec plusieurs arguments
85 #
86 # Une fonction peut recevoir plusieurs nombres : on les sépare par des
87 # virgules. Cette fonction regarde si le nombre a divise le nombre b.
88
89 def divise(a, b):
90     if b % a == 0:
91         return("a divise b")
92     else:
93         return("a ne divise pas b")
94
95 # Tests à taper dans la console :
96 #     divise(3, 12)                             réponse attendue : 'a divise b'
97 #     divise(5, 12)                             réponse attendue : 'a ne divise pas b'
98 #
99 # [QUESTION 2] Tapez divise(12, 3) : pourquoi la réponse change-t-elle ?
100 # Réponse :
101
102
103 # %% [EXEMPLE 4] Afficher plusieurs résultats avec print
104 #
105 # print(...) AFFICHE dans la console. On peut afficher plusieurs choses
106 # à la suite, en les séparant par des virgules.
107 # a // b donne le QUOTIENT de la division de a par b (sans la virgule).
108
109 def division(a, b):
110     q = a // b
111     r = a % b
112     print("quotient :", q, " reste :", r)
113
114 # Tests à taper dans la console :
115 #     division(17, 3)                            réponse attendue : quotient : 5   reste : 2
116 #     division(20, 4)                            réponse attendue : quotient : 5   reste : 0
117
118
119 # %% [EXEMPLE 5] Un « si » dans un « si »

```

```

120 #
121 # Pour vérifier DEUX conditions, on peut placer un if à l'intérieur d'un
122 # autre if. Attention au décalage vers la droite (l'indentation) : c'est
123 # lui qui indique à Python ce qui est « à l'intérieur ».
124
125 def divisible_par_3_et_5(n):
126     if n % 3 == 0:
127         if n % 5 == 0:
128             return("divisible par 3 et par 5")
129         else:
130             return("divisible par 3 mais pas par 5")
131     else:
132         return("pas divisible par 3")
133
134 # Tests à taper dans la console :
135 # divisible_par_3_et_5(30)           réponse attendue : 'divisible par 3 et par 5'
136 # divisible_par_3_et_5(9)           réponse attendue : 'divisible par 3 mais pas par 5'
137 # divisible_par_3_et_5(10)          réponse attendue : 'pas divisible par 3'
138 #
139 # [QUESTION 3] Modifiez la fonction pour que divisible_par_3_et_5(10)
140 # réponde "divisible par 5 mais pas par 3". (Il faut ajouter un if dans
141 # le else.)
142
143
144 # =====
145 # PLACE À LA GÉOMÉTRIE !
146 # Dans toute la suite, on se place dans un repère orthonormé.
147 # Un point A(xA ; yA) est donné par DEUX variables : xA et yA.
148 # ATTENTION : en Python, la virgule décimale s'écrit avec un POINT :
149 # 2,5 s'écrit 2.5 (la virgule sert à séparer les nombres).
150 # =====
151
152
153 # %% PARTIE 1 - [À COMPLÉTER] Milieu d'un segment
154 #
155 # La fonction milieu reçoit les coordonnées de A et de B et AFFICHE les
156 # coordonnées du milieu du segment [AB].
157
158 def milieu(xA, yA, xB, yB):
159     x = ...
160     y = ...
161     print("Le milieu a pour coordonnées :", x, ";", y)
162
163 # Tests à taper dans la console :
164 # milieu(1, 2, 5, 4)           réponse attendue : Le milieu a pour coordonnées : 3.0
165 #                               ↪ ; 3.0
166 # milieu(-2, 1, 3, 0)          réponse attendue : Le milieu a pour coordonnées : 0.5
167 #                               ↪ ; 0.5
168 # milieu(4, -6, 4, 6)          réponse attendue : Le milieu a pour coordonnées : 4.0
169 #                               ↪ ; 0.0
170
171
172 # %% PARTIE 2 - [À COMPLÉTER] Distance entre deux points
173 #
174 # La fonction distance reçoit les coordonnées de A et de B et renvoie la
175 # distance AB.
176 # En Python :  $x^2$  s'écrit  $x**2$  et la racine carrée de x s'écrit  $\sqrt{x}$ .
177 # La ligne from math import sqrt permet d'utiliser sqrt : ne l'effacez pas.
178
179 from math import sqrt
180
181 def distance(xA, yA, xB, yB):

```

```

179     d = ...
180     return(d)
181
182 # Tests à taper dans la console :
183 #   distance(1, 1, 4, 5)                réponse attendue : 5.0
184 #   distance(-3, 2, 5, 2)              réponse attendue : 8.0
185 #   distance(0, 0, 1, 1)              réponse attendue : 1.4142135623730951
186 #
187 # [QUESTION 4] Quelle est la valeur exacte de la distance entre les points
188 # (0 ; 0) et (1 ; 1) ? Python donne-t-il cette valeur exacte ?
189 # Réponse :
190
191
192 # %% PARTIE 2 - [EXEMPLE 6] Attention aux nombres à virgule !
193 #
194 # Un ordinateur garde un nombre LIMITÉ de chiffres après la virgule : il ne
195 # connaît qu'une VALEUR APPROCHÉE de racine de 2. Tapez dans la console :
196 #   sqrt(2) ** 2                        on attend 2... que répond Python ?
197 #   0.1 + 0.2                          on attend 0.3... que répond Python ?
198 #
199 # CONCLUSION : si on compare deux distances avec ==, Python peut se tromper.
200 # L'astuce : une longueur est positive, donc  $AB = CD$  revient à  $AB^2 = CD^2$ .
201 # Et  $AB^2$  se calcule SANS racine carrée : avec des coordonnées entières,
202 # Python le calcule de façon EXACTE.
203
204
205 # %% PARTIE 2 - [À COMPLÉTER] Carré de la distance
206 #
207 # La fonction distance_carre renvoie  $AB^2$  (le carré de la distance AB),
208 # sans utiliser de racine carrée.
209
210 def distance_carre(xA, yA, xB, yB):
211     d2 = ...
212     return(d2)
213
214 # Tests à taper dans la console :
215 #   distance_carre(1, 1, 4, 5)          réponse attendue : 25
216 #   distance_carre(0, 0, 1, 1)          réponse attendue : 2
217 #   distance_carre(2, -1, -1, 3)        réponse attendue : 25
218 #
219 # RÈGLE POUR LA SUITE DU TP : pour savoir si deux longueurs sont égales,
220 # on compare TOUJOURS leurs carrés.
221
222
223 # %% LES FIGURES DE TEST
224 #
225 # Dans toute la suite, ABCD est un quadrilatère dont les sommets sont donnés
226 # DANS L'ORDRE (on tourne autour de la figure).
227 # Les fonctions recevront 8 nombres, toujours dans cet ordre :
228 #     xA, yA, xB, yB, xC, yC, xD, yD
229 #
230 #   Figure 1 : A(0 ; 0)   B(4 ; 1)   C(5 ; 3)   D(1 ; 2)
231 #   Figure 2 : A(1 ; 1)   B(5 ; 1)   C(5 ; 3)   D(1 ; 3)
232 #   Figure 3 : A(0 ; 0)   B(2 ; 2)   C(-1 ; 5)   D(-3 ; 3)
233 #   Figure 4 : A(0 ; 0)   B(3 ; 1)   C(4 ; 4)   D(1 ; 3)
234 #   Figure 5 : A(0 ; 0)   B(2 ; 0)   C(2 ; 2)   D(0 ; 2)
235 #   Figure 6 : A(1 ; 0)   B(4 ; 1)   C(3 ; 4)   D(0 ; 3)
236 #   Figure 7 : A(0 ; 0)   B(5 ; 0)   C(4 ; 3)   D(1 ; 2)
237 #   Figure 8 : A(0 ; 0)   B(4 ; 1)   C(1 ; 2)   D(5 ; 3)
238
239
240 # %% PARTIE 3 - [À COMPLÉTER] Parallélogramme

```

```

241 #
242 # La fonction renvoie un message qui dit si ABCD est un parallélogramme.
243 # (Revoyez l'exemple 5 : un « si » dans un « si ».)
244
245 def parallelogramme(xA, yA, xB, yB, xC, yC, xD, yD):
246     # coordonnées du milieu de [AC]
247     x1 = (xA + xC) / 2
248     y1 = ...
249     # coordonnées du milieu de [BD]
250     x2 = ...
251     y2 = ...
252     if x1 == x2:
253         if ... :
254             return("ABCD est un parallélogramme")
255         else:
256             return("ABCD n'est pas un parallélogramme")
257     else:
258         return(...)
259
260 # Tests à taper dans la console :
261 # parallelogramme(0, 0, 4, 1, 5, 3, 1, 2)    réponse attendue : 'ABCD est un parallélogramme'
262 # parallelogramme(0, 0, 2, 2, -1, 5, -3, 3)  réponse attendue : 'ABCD est un parallélogramme'
263 # parallelogramme(0, 0, 5, 0, 4, 3, 1, 2)    réponse attendue : "ABCD n'est pas un
↳ parallelogramme"
264 # parallelogramme(0, 0, 4, 1, 1, 2, 5, 3)    réponse attendue : "ABCD n'est pas un
↳ parallelogramme"
265 #
266 # [QUESTION 5] Les figures 1 et 8 ont les mêmes points. Pourquoi la figure 8
267 # n'est-elle pas un parallélogramme ? (Tracez-la !)
268 # Réponse :
269
270
271 # %% PARTIE 4 - [À ÉCRIRE] Losange
272 #
273 # Écrivez la fonction losange(xA, yA, xB, yB, xC, yC, xD, yD) qui renvoie
274 # "ABCD est un losange" ou "ABCD n'est pas un losange".
275 # Aide : partez de votre fonction parallelogramme et ajoutez ce qu'il faut.
276 # Pour comparer deux longueurs, calculez leurs carrés.
277
278 # Écrivez votre fonction ici :
279
280
281
282 # Tests à taper dans la console :
283 # losange(0, 0, 3, 1, 4, 4, 1, 3)            réponse attendue : 'ABCD est un losange'
284 # losange(1, 0, 4, 1, 3, 4, 0, 3)            réponse attendue : 'ABCD est un losange'
285 # losange(0, 0, 4, 1, 5, 3, 1, 2)            réponse attendue : "ABCD n'est pas un losange"
286 # losange(1, 1, 5, 1, 5, 3, 1, 3)            réponse attendue : "ABCD n'est pas un losange"
287
288
289 # %% PARTIE 5 - [À ÉCRIRE] Rectangle
290 #
291 # Écrivez la fonction rectangle(xA, yA, xB, yB, xC, yC, xD, yD) qui
292 # renvoie "ABCD est un rectangle" ou "ABCD n'est pas un rectangle".
293
294 # Écrivez votre fonction ici :
295
296
297
298 # Tests à taper dans la console :
299 # rectangle(1, 1, 5, 1, 5, 3, 1, 3)          réponse attendue : 'ABCD est un rectangle'
300 # rectangle(0, 0, 2, 2, -1, 5, -3, 3)        réponse attendue : 'ABCD est un rectangle'

```

```

301 # rectangle(0, 0, 4, 1, 5, 3, 1, 2)          réponse attendue : "ABCD n'est pas un rectangle"
302 # rectangle(0, 0, 3, 1, 4, 4, 1, 3)          réponse attendue : "ABCD n'est pas un rectangle"
303 #
304 # [QUESTION 6] Dans la figure 3, calculez à la main la longueur exacte des
305 # diagonales. Pourquoi a-t-on eu raison de comparer les carrés ?
306 # Réponse :
307
308
309 # %% PARTIE 6 - [À ÉCRIRE] Carré
310 #
311 # Écrivez la fonction carre(xA, yA, xB, yB, xC, yC, xD, yD) (sans accent
312 # dans le nom !) qui renvoie "ABCD est un carré" ou "ABCD n'est pas un carré".
313
314 # Écrivez votre fonction ici :
315
316
317
318 # Tests à taper dans la console :
319 # carre(0, 0, 2, 0, 2, 2, 0, 2)              réponse attendue : 'ABCD est un carré'
320 # carre(1, 0, 4, 1, 3, 4, 0, 3)              réponse attendue : 'ABCD est un carré'
321 # carre(1, 1, 5, 1, 5, 3, 1, 3)              réponse attendue : "ABCD n'est pas un carré"
322 # carre(0, 0, 3, 1, 4, 4, 1, 3)              réponse attendue : "ABCD n'est pas un carré"
323
324
325 # %% PARTIE 7 - [À ÉCRIRE] Nature exacte du quadrilatère
326 #
327 # Écrivez la fonction nature(xA, yA, xB, yB, xC, yC, xD, yD) qui renvoie
328 # la nature la plus précise de ABCD :
329 # "ABCD est un carré", "ABCD est un rectangle", "ABCD est un losange",
330 # "ABCD est un parallélogramme" ou "ABCD est un quadrilatère quelconque".
331
332 # Écrivez votre fonction ici :
333
334
335
336 # Tests à taper dans la console :
337 # nature(0, 0, 4, 1, 5, 3, 1, 2)              réponse attendue : 'ABCD est un parallélogramme'
338 # nature(1, 1, 5, 1, 5, 3, 1, 3)              réponse attendue : 'ABCD est un rectangle'
339 # nature(0, 0, 2, 2, -1, 5, -3, 3)            réponse attendue : 'ABCD est un rectangle'
340 # nature(0, 0, 3, 1, 4, 4, 1, 3)              réponse attendue : 'ABCD est un losange'
341 # nature(0, 0, 2, 0, 2, 2, 0, 2)              réponse attendue : 'ABCD est un carré'
342 # nature(1, 0, 4, 1, 3, 4, 0, 3)              réponse attendue : 'ABCD est un carré'
343 # nature(0, 0, 5, 0, 4, 3, 1, 2)              réponse attendue : 'ABCD est un quadrilatère
    ↪ quelconque'
344 # nature(0, 0, 4, 1, 1, 2, 5, 3)              réponse attendue : 'ABCD est un quadrilatère
    ↪ quelconque'
345 #
346 # [QUESTION 7] La figure 6 est aussi un losange et un rectangle. Pourquoi
347 # votre fonction nature répond-elle seulement « carré » ?
348 # Réponse :
349
350
351 # %% DÉFI - À vous de jouer !
352 #
353 # 1) On considère les points E(-2 ; 1), F(1 ; -1), G(3 ; 2) et H(0 ; 4).
354 # Quelle est la nature du quadrilatère EFGH ? Répondez avec votre
355 # programme, puis justifiez sur papier.
356 # Réponse :
357 #
358 # 2) Inventez un losange qui n'est pas un carré, et vérifiez-le avec nature.
359 # Réponse :
360

```

```
361
362 # %% POUR ALLER PLUS LOIN
363 #
364 # Écrivez une fonction perimetre(xA, yA, xB, yB, xC, yC, xD, yD) qui
365 # renvoie le périmètre du quadrilatère ABCD.
366 # Tests : figure 2 -> 12.0      figure 5 -> 8.0
```